TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP GIỮA KỲ

Môn: Lập trình ứng dụng di động

THIẾT KẾ VÀ XÂY DỰNG CHỨC NĂNG MÃ GIẢM GIÁ (VOUCHER) TRONG HỆ THỐNG ĐẶT ĐỒ ĂN FOODGO

Họ và tên sinh viên: Trần Minh Mẫn
Mã số sinh viên: 25TX810012
Giảng viên hướng dẫn: Nguyễn Thủy An

Tháng 11, 2026

Mục lục

1 Giới thiệu **2**
1.1 Bối cảnh đồ án 2
1.2 Lý do chọn đề tài 2

2 Phân tích yêu cầu **3**
2.1 Yêu cầu chức năng 3
2.2 Yêu cầu phi chức năng 4

3 Thiết kế hệ thống **4**
3.1 Thiết kế cơ sở dữ liệu 4
3.2 Thiết kế API 5
3.3 Luồng xử lý khi đặt hàng có áp dụng mã giảm giá 5
3.4 Cơ chế chống sử dụng vượt giới hạn khi có nhiều yêu cầu đồng thời 6

4 Cài đặt **7**
4.1 Phía backend 7
4.2 Phía ứng dụng Android 8

5 Kiểm thử **9**
5.1 Kiểm thử đơn vị (unit test) 9
5.2 Kiểm thử tích hợp với cơ sở dữ liệu thật 9
5.3 Kết quả chạy toàn bộ bộ kiểm thử 10
5.4 Kiểm thử thủ công qua HTTP thật 10

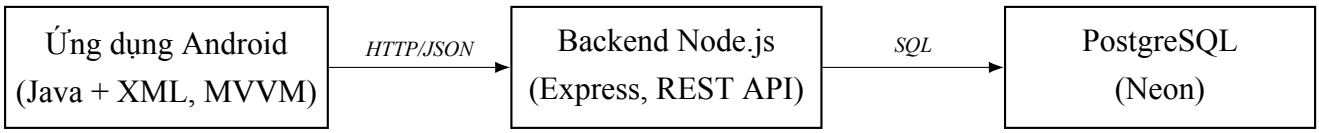
6 Kết luận và hướng phát triển **14**
6.1 Kết quả đạt được 14
6.2 Hạn chế 14
6.3 Hướng phát triển 14

1 Giới thiệu

1.1 Bối cảnh đồ án

FoodGo là ứng dụng đặt đồ ăn trực tuyến cho một nhà hàng duy nhất, gồm hai phần:

- **Ứng dụng di động Android** viết bằng Java, giao diện XML, theo mô hình MVVM (gồm Activity/Fragment, ViewModel và Repository).
- **Backend** viết bằng Node.js (Express), cơ sở dữ liệu PostgreSQL (triển khai trên Neon), giao tiếp với app qua REST API dạng JSON.



Hình 1: Kiến trúc tổng quan của hệ thống FoodGo

Tại thời điểm thực hiện báo cáo này, hệ thống đã hoàn thiện các chức năng: đăng ký/đăng nhập, xem thực đơn, tìm kiếm bằng AI (rule-based kết hợp LLM dự phòng), giỏ hàng, đặt hàng, thanh toán qua cổng VNPay và ZaloPay, đánh giá món ăn, chatbot AI hỗ trợ khách hàng, bản đồ hiển thị vị trí giao hàng theo thời gian thực, và tính năng ước lượng thời gian giao hàng thông minh (Smart ETA). Tuy nhiên, hệ thống **chưa có** chức năng mã giảm giá (voucher) – một chức năng phổ biến và có giá trị nghiệp vụ cao đối với mọi ứng dụng thương mại điện tử/đặt hàng trực tuyến.

1.2 Lý do chọn đề tài

Sau khi khảo sát toàn bộ mã nguồn của dự án (phần backend và phần Android), nhóm thực hiện xác nhận chức năng mã giảm giá hoàn toàn chưa tồn tại dưới bất kỳ hình thức nào: không có bảng dữ liệu, không có API, không có màn hình liên quan. Đây là chức năng được lựa chọn để triển khai hoàn chỉnh cho báo cáo giữa kỳ, vì các lý do sau:

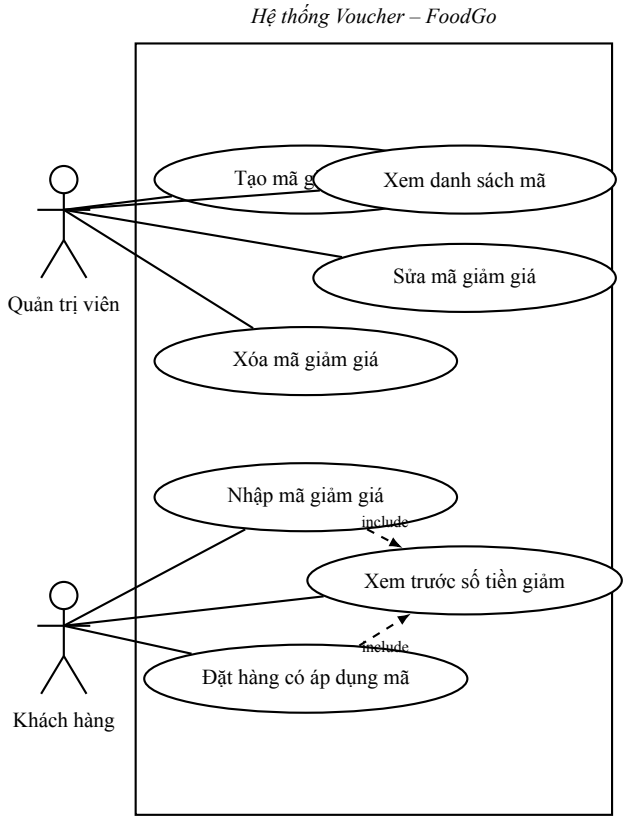
1. Gắn trực tiếp vào luồng nghiệp vụ quan trọng nhất của hệ thống (đặt hàng và thanh toán), nên thể hiện được khả năng tích hợp vào một hệ thống có sẵn thay vì xây dựng một module độc lập.
2. Có độ phức tạp kỹ thuật vừa đủ để trình bày các khía cạnh thiết kế quan trọng: thiết kế cơ sở dữ liệu quan hệ, thiết kế API RESTful, và đặc biệt là **xử lý tương tranh (concurrency control)** để chống việc một mã giảm giá bị sử dụng vượt quá giới hạn cho phép khi có nhiều yêu cầu đặt hàng đến cùng lúc.
3. Có đầy đủ hai vai trò người dùng (Quản trị viên tạo/quản lý mã; Khách hàng áp dụng mã), phù hợp để trình bày cả hai phía của một hệ thống hoàn chỉnh.

2 Phân tích yêu cầu

2.1 Yêu cầu chức năng

Vai trò	Yêu cầu
Quản trị viên	Tạo, xem danh sách, sửa, xóa mã giảm giá. Mỗi mã có thể là giảm theo *phần trăm* (kèm mức giảm tối đa tùy chọn) hoặc giảm *số tiền cố định*; có thể đặt điều kiện giá trị đơn hàng tối thiểu, giới hạn tổng số lượt sử dụng, giới hạn số lượt sử dụng trên mỗi khách hàng, thời gian hiệu lực (từ ngày – đến ngày), và trạng thái kích hoạt/khóa.
Khách hàng	Nhập mã giảm giá tại màn hình thanh toán; hệ thống kiểm tra và hiển thị trước số tiền được giảm. Khi đặt hàng, số tiền giảm giá được áp dụng vào tổng tiền thanh toán (COD, VNPay hoặc ZaloPay).

Bảng 1: Yêu cầu chức năng theo vai trò



Hình 2: Sơ đồ use-case chức năng voucher

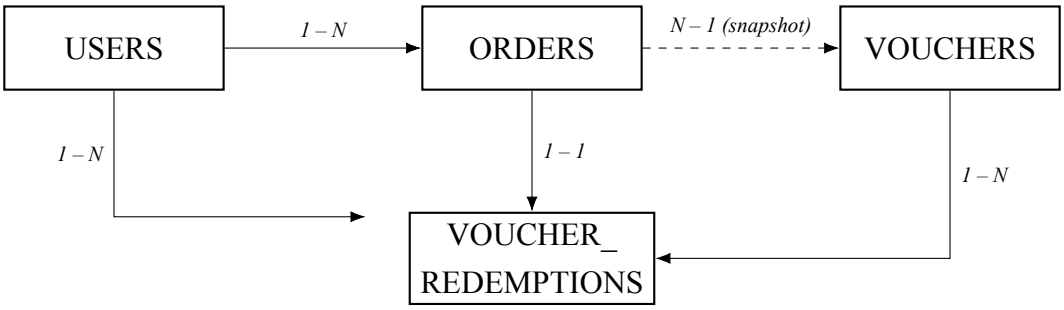
2.2 Yêu cầu phi chức năng

Yêu cầu quan trọng nhất về mặt kỹ thuật là **tính đúng đắn khi có nhiều giao dịch đồng thời**: khi một mã giảm giá được giới hạn tổng số lượt sử dụng (ví dụ chỉ 1 lượt), hệ thống phải đảm bảo không có hai đơn hàng cùng được áp dụng mã đó nếu hai yêu cầu đặt hàng đến *đúng cùng một thời điểm*. Đây là một lỗi *race condition* điển hình nếu chỉ kiểm tra điều kiện bằng một câu lệnh `SELECT` rồi mới `INSERT` mà không có cơ chế khóa dữ liệu phù hợp.

3 Thiết kế hệ thống

3.1 Thiết kế cơ sở dữ liệu

Hai bảng mới được bổ sung vào cơ sở dữ liệu PostgreSQL sẵn có: `vouchers` (định nghĩa mã giảm giá) và `voucher_redemptions` (sổ ghi nhận các lượt sử dụng mã, dùng làm nguồn dữ liệu duy nhất để đếm số lượt đã dùng). Bảng `orders` được bổ sung hai cột `voucher_id` và `discount_amount` để lưu lại (snapshot) mã và số tiền đã giảm của từng đơn hàng.



Hình 3: Sơ đồ quan hệ giữa các bảng liên quan đến voucher

Cột	Kiểu	Ý nghĩa
code	citext, unique	Mã giảm giá, không phân biệt hoa/thường
discount_type	text	`percent` hoặc `fixed`
discount_value	bigint	Giá trị giảm (% hoặc VNĐ tuỳ loại)
max_discount_amount	bigint, null	Mức giảm tối đa (chỉ áp dụng cho loại `percent`)
min_order_value	bigint	Giá trị đơn hàng tối thiểu để áp dụng mã
usage_limit	integer, null	Tổng số lượt được sử dụng (null = không giới hạn)
usage_limit_per_user	integer, null	Số lượt tối đa mỗi khách hàng được dùng
valid_from / valid_until	timestamptz, null	Thời gian hiệu lực
is_active	boolean	Trạng thái kích hoạt/khóa

Bảng 2: Cấu trúc bảng `vouchers`

Cột	Kiểu	Ý nghĩa
voucher_id	uuid, FK	Mã giảm giá đã áp dụng
user_id	uuid, FK	Khách hàng đã sử dụng
order_id	uuid, FK, unique	Đơn hàng liên quan (1 đơn chỉ áp 1 mã)
discount_amount	bigint	Số tiền thực tế đã giảm cho đơn này

Bảng 3: Cấu trúc bảng `voucher_redemptions` (sổ ghi nhận, chỉ thêm, không sửa/xóa)

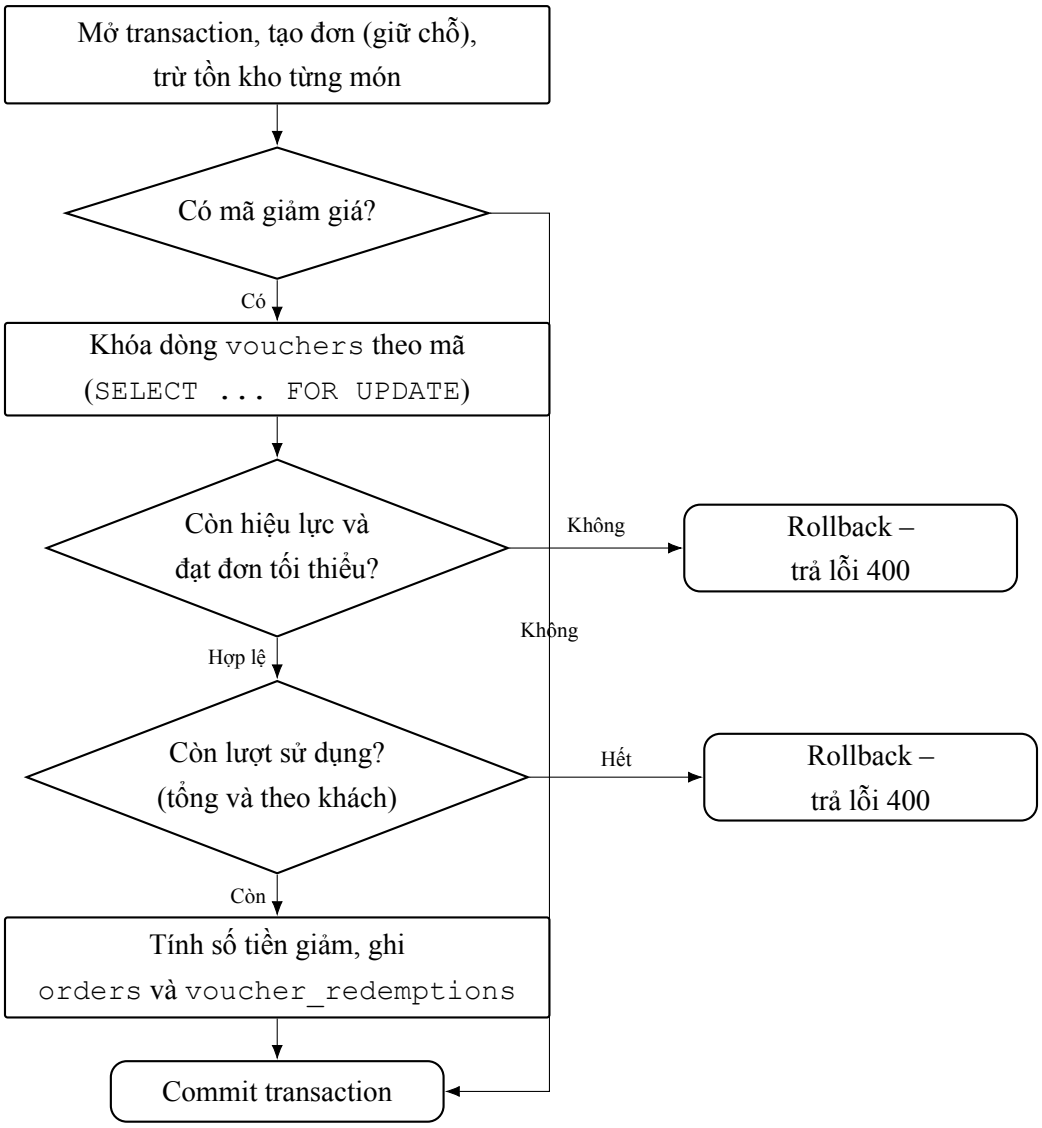
3.2 Thiết kế API

Method	Endpoint	Quyền	Mô tả
GET	/api/voucher	Admin	Danh sách mã giảm giá
GET	/api/voucher/:id	Admin	Chi tiết một mã
POST	/api/voucher	Admin	Tạo mã mới
PUT	/api/voucher/:id	Admin	Cập nhật mã
DELETE	/api/voucher/:id	Admin	Xóa mã (từ chối nếu đã có lượt sử dụng)
POST	/api/voucher/validate	Khách hàng	Xem trước số tiền được giảm (không ghi dữ liệu)
POST	/api/order	Khách hàng	Đặt hàng – nhận thêm trường `voucherCode` (tuỳ chọn)

Bảng 4: Danh sách API của chức năng voucher

3.3 Luồng xử lý khi đặt hàng có áp dụng mã giảm giá

Toàn bộ việc kiểm tra và áp dụng mã giảm giá diễn ra **bên trong transaction đặt hàng** (cùng với việc trừ tồn kho và tạo đơn), để đảm bảo nếu có lỗi ở bất kỳ bước nào, toàn bộ thao tác được hoàn tác (rollback) – không để lại đơn hàng hay dữ liệu mồ côi.



Hình 4: Luồng xử lý áp dụng mã giảm giá trong transaction đặt hàng

3.4 Cơ chế chống sử dụng vượt giới hạn khi có nhiều yêu cầu đồng thời

Nếu chỉ đơn giản đếm số lượt đã dùng rồi so sánh với giới hạn trước khi ghi thêm một lượt mới, hai yêu cầu đặt hàng đến gần như đồng thời có thể *cùng đọc* được số lượt hiện tại (ví dụ cả hai đều đọc được "đã dùng 0/1 lượt"), rồi *cả hai cùng ghi* thêm một lượt sử dụng – kết quả là mã giảm giá bị dùng vượt giới hạn cho phép.

Giải pháp áp dụng trong hệ thống là dùng câu lệnh `SELECT ... FOR UPDATE` để khóa đúng dòng dữ liệu của mã giảm giá đó ngay khi bắt đầu kiểm tra, trong cùng transaction với toàn bộ việc đặt hàng:

Listing 1: Khóa dòng voucher trước khi kiểm tra (backend/db/vouchers.js)

```
SELECT id, code, discount_type, discount_value, ...
FROM vouchers WHERE code = $1 FOR UPDATE
```

Với cách này, nếu hai giao dịch đặt hàng cùng dùng một mã giảm giá đến cùng lúc, giao dịch thứ hai sẽ phải **chờ** giao dịch thứ nhất hoàn tất (commit hoặc rollback) mới được tiếp tục đọc dòng dữ liệu đó. Nhờ vậy, việc ”đếm số lượt đã dùng rồi ghi thêm một lượt mới” luôn diễn ra tuần tự (serialize) đối với cùng một mã giảm giá, loại bỏ hoàn toàn khả năng xảy ra race condition. Mục 5.2 trình bày test case thực nghiệm chứng minh cơ chế này hoạt động đúng trên cơ sở dữ liệu PostgreSQL thật.

4 Cài đặt

4.1 Phía backend

Hàm thuần (không phụ thuộc cơ sở dữ liệu) tính số tiền được giảm, dùng chung cho cả API xem trước (`validate`) và API đặt hàng thật:

Listing 2: backend/util/voucher.js – calcDiscountAmount

```
export const calcDiscountAmount = (voucher, orderAmount) => {
  let discount;
  if (voucher.discountType === "percent") {
    discount = Math.floor((orderAmount * voucher.discountValue) /
       100);
    if (voucher.maxDiscountAmount != null) {
      discount = Math.min(discount, voucher.maxDiscountAmount);
    }
  } else {
    discount = voucher.discountValue;
  }
  return Math.max(0, Math.min(discount, orderAmount));
};
```

Đoạn xử lý chính trong transaction đặt hàng (rút gọn), nằm trong hàm `placeOrder` của `backend/controllers/orderController.js`:

Listing 3: Tích hợp voucher vào placeOrder (rút gọn)

```
if (voucherCodeRaw) {
  const voucher = await lockVoucherByCode(client, voucherCodeRaw);
  const problem = validateVoucherConditions(voucherForCalc, amount);
  if (problem) throw new HttpError(400, problem);

  const totalUsed = await countVoucherRedemptions(client, voucher.id)
     ;
  if (voucher.usage_limit != null && totalUsed >= voucher.usage_limit
     ) {
```

```
    throw new HttpError(400, "Mã ảgim giá đã ết uợlt ửs ụdng");
  }
  discountAmount = calcDiscountAmount(voucherForCalc, amount);
  finalAmount = amount - discountAmount;
  voucherIdApplied = voucher.id;
}
await applyOrderVoucher(client, orderId, { amount: finalAmount,
   discountAmount, voucherId: voucherIdApplied });
```

4.2 Phía ứng dụng Android

Về phía quản trị viên, màn hình `AdminVoucherActivity` hiển thị danh sách mã giảm giá (tạo/sửa/xóa), theo đúng kiến trúc MVVM đã dùng xuyên suốt ứng dụng (`ViewModel` giao tiếp với `Repository`, `Repository` gọi `Retrofit`). Về phía khách hàng, màn hình `CheckoutActivity` được bổ sung một ô nhập mã giảm giá; khi khách hàng bấm ”Áp dụng”, `CheckoutViewModel` gọi API xem trước:

Listing 4: CheckoutViewModel.applyVoucherCode (rút gọn)

```
public void applyVoucherCode(String code) {
   String trimmed = code != null ? code.trim() : "";
   if (trimmed.isEmpty()) {
      appliedVoucherCode.setValue(null);
      discountAmount.setValue(0.0);
      return;
   }
   voucherRepository.validateVoucher(trimmed, totalAmount.getValue()
      )
      .enqueue(new Callback<ValidateVoucherResponse>() {
         public void onResponse(..., Response<
            ValidateVoucherResponse> response) {
            // thanh cong: luu ma + so tien giam de hien thi va de
               gui
            // kem khi dat hang thuc su (xem placeOrder())
         }
      });
}
```

Khi khách hàng bấm ”Đặt hàng”, mã giảm giá đã áp dụng (nếu có) được gửi kèm trong trường `voucherCode` của yêu cầu tạo đơn hàng; việc kiểm tra và áp dụng *thật sự* (có khóa dữ liệu, chống trùng lặp) diễn ra hoàn toàn ở phía backend như trình bày ở Mục 3.4 – phía Android chỉ hiển thị kết quả xem trước để cải thiện trải nghiệm người dùng.

5 Kiểm thử

5.1 Kiểm thử đơn vị (unit test)

File `backend/test/voucher.test.js` chứa 16 test case kiểm tra các hàm xử lý thuần (tính số tiền giảm, kiểm tra điều kiện áp dụng, kiểm tra định dạng mã, kiểm tra quy tắc nghiệp vụ khi tạo mã), không phụ thuộc cơ sở dữ liệu.

5.2 Kiểm thử tích hợp với cơ sở dữ liệu thật

File `backend/test/voucher.db.test.js` chứa 9 test case chạy trực tiếp trên PostgreSQL thật (không dùng mock), bao gồm các tình huống trong Bảng 5.

#	Tình huống kiểm thử
1	Áp mã giảm theo % có mức giảm tối đa – số tiền giảm bị giới hạn đúng bằng mức tối đa
2	Áp mã giảm số tiền cố định lớn hơn giá trị đơn hàng – số tiền giảm bị giới hạn lại bằng giá trị đơn hàng (không âm tiền)
3	Đơn hàng dưới giá trị tối thiểu – bị từ chối, **toàn bộ transaction được hoàn tác** (không mất tồn kho, không tạo đơn mồ côi)
4	Mã đã hết hạn hoặc đang bị khóa – bị từ chối với thông báo phù hợp
5	Giới hạn tổng số lượt sử dụng (toàn hệ thống) – đơn hàng vượt giới hạn bị từ chối
6	Giới hạn số lượt sử dụng theo từng khách hàng – không ảnh hưởng đến khách hàng khác
7	**Hai yêu cầu đặt hàng gọi đồng thời** (dùng `Promise.all`) cùng một mã có giới hạn 1 lượt, từ hai món ăn và hai khách hàng khác nhau (để không bị ”khóa nhờ” bởi khóa tồn kho) – kết quả: đúng một yêu cầu thành công, một bị từ chối, bảng ghi nhận lượt dùng chỉ có đúng một dòng
8	Không dùng mã giảm giá – hành vi cũ (trước khi có voucher) không bị thay đổi
9	Xóa mã giảm giá đã có người dùng – bị từ chối (409), chỉ có thể khóa (vô hiệu hóa)

Bảng 5: Danh sách test case tích hợp với cơ sở dữ liệu thật

Test case số 7 được chạy lại 5 lần liên tiếp để xác nhận không xảy ra hiện tượng không ổn định (flaky) do yếu tố thời gian. Kết quả toàn bộ 5 lần chạy đều cho đúng 1 giao dịch thành công

và đúng 1 dòng trong bảng `voucher_redemptions`, xác nhận cơ chế khóa `SELECT ... FOR UPDATE` hoạt động đúng như thiết kế.

5.3 Kết quả chạy toàn bộ bộ kiểm thử

Bộ test	**Số test case**	**Pass**	**Fail**
Bộ test có sẵn của dự án (trước khi thêm voucher)	226	226	0
Test đơn vị cho voucher (`voucher.test.js`)	16	16	0
Test tích hợp DB cho voucher (`voucher.db.test.js`)	9	9	0
Tổng	**251**	**251**	**0**

Bảng 6: Kết quả chạy `npm test` sau khi hoàn thiện chức năng voucher

Việc bộ test có sẵn của dự án (226 test case, bao phủ các chức năng đặt hàng, thanh toán VNPay/ZaloPay, Smart ETA, v.v.) vẫn pass toàn bộ sau khi thêm chức năng voucher xác nhận thay đổi không gây ảnh hưởng ngoài ý muốn (regression) đến các chức năng đã có trước đó.

Phía ứng dụng Android, mã nguồn biên dịch thành công bằng Gradle, không phát sinh lỗi cú pháp. Toàn bộ bộ test đơn vị hiện có (chạy trên Robolectric) vẫn pass sau khi bổ sung trường `voucherCode`, bao gồm 5 test case của `CheckoutViewModelTest` kiểm tra luồng đặt hàng.

5.4 Kiểm thử thủ công qua HTTP thật

Do môi trường thực hiện báo cáo không có sẵn máy giả lập (emulator) Android, nhóm thực hiện kiểm thử luồng hoạt động thật của API bằng cách khởi chạy server backend thật (kết nối cơ sở dữ liệu PostgreSQL cục bộ) và gọi API bằng `curl`. Dưới đây là một kết quả thực tế: tạo mã giảm giá cố định 20.000đ, sau đó đặt một đơn hàng COD trị giá 50.000đ có áp dụng mã (viết thường `order20`, hệ thống tự nhận diện không phân biệt hoa/thường):

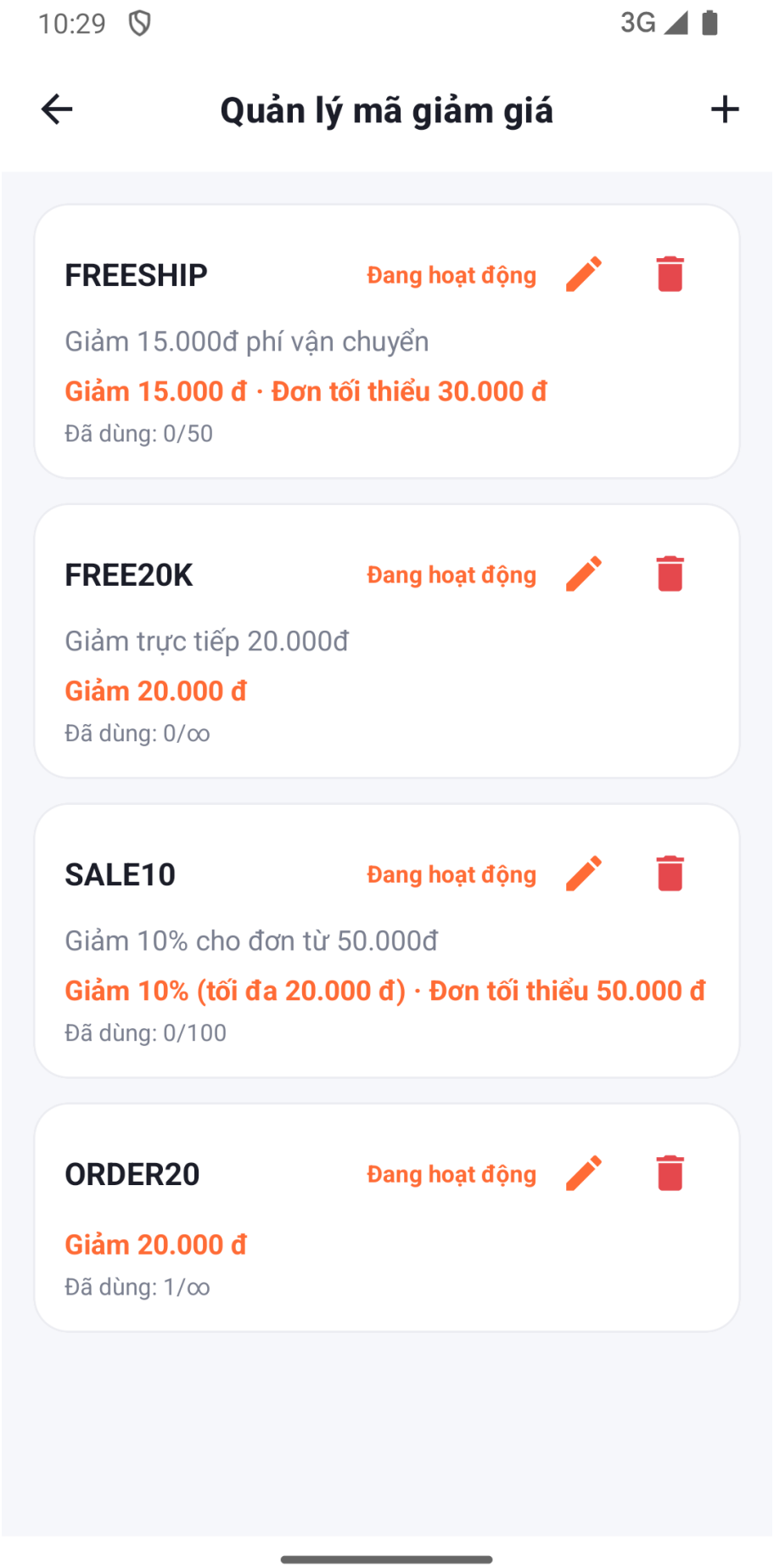
Listing 5: Kết quả gọi API thật – đặt hàng có áp dụng voucher

```
{
  "success": true,
  "orderId": "fff81e39-...",
  "order": {
    "amount": 30000,
    "discountAmount": 20000,
    "voucherCode": "ORDER20",
    "status": "pending",
    "paymentMethod": "cod"
    ...
```

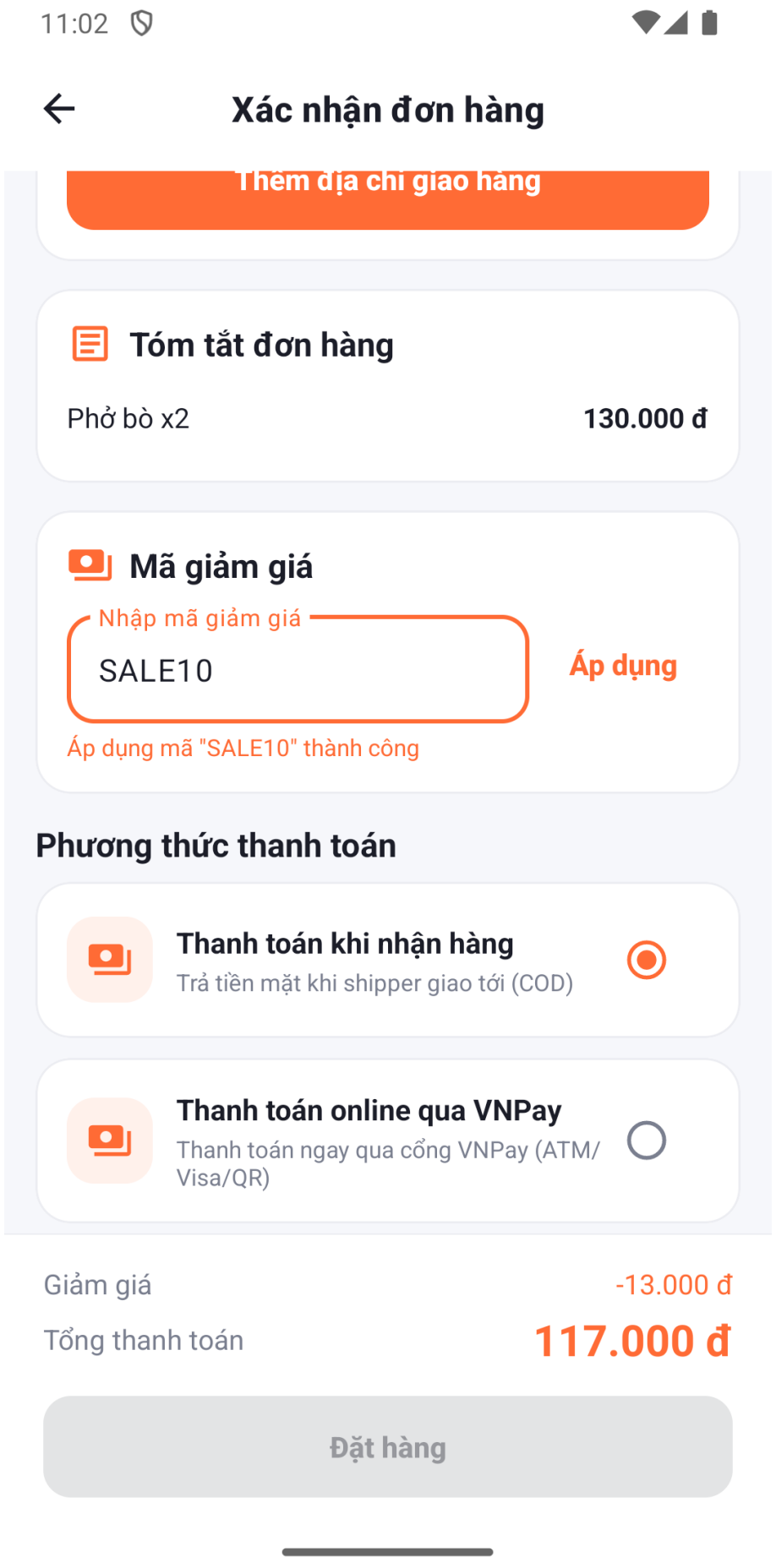
```
  }
}
```

Kết quả cho thấy tổng tiền cuối cùng (`amount`) đã được trừ đúng 20.000đ so với giá gốc 50.000đ, và mã giảm giá được ghi nhận lại (`voucherCode`) để hiển thị trong lịch sử đơn hàng.

Ngoài kiểm thử qua API, chức năng cũng đã được chạy thật trên máy giả lập Android (Pixel 3a, Android 14) với ứng dụng FoodGo cài đặt từ bản build Debug, kết nối tới server backend thật. Hình 5 là màn hình ”Quản lý mã giảm giá” dành cho quản trị viên, hiển thị danh sách các mã đã tạo; Hình 6 là màn hình Checkout của khách hàng sau khi nhập mã `SALE10` và bấm ”Áp dụng” – hệ thống xác nhận thành công và cập nhật lại số tiền giảm giá cùng tổng thanh toán.



Hình 5: Màn hình Quản lý mã giảm giá (Admin)



Hình 6: Màn hình Checkout – áp dụng mã giảm giá (Khách hàng)

6 Kết luận và hướng phát triển

6.1 Kết quả đạt được

Báo cáo đã trình bày việc phân tích, thiết kế và xây dựng hoàn chỉnh chức năng mã giảm giá cho hệ thống FoodGo, bao gồm: thiết kế cơ sở dữ liệu quan hệ, thiết kế và cài đặt API RESTful, cài đặt giao diện quản trị và giao diện khách hàng trên Android, và đặc biệt là giải quyết đúng đắn bài toán xử lý tương tranh khi nhiều yêu cầu cùng sử dụng một mã giảm giá có giới hạn số lượt dùng – được kiểm chứng bằng test case thực nghiệm trên cơ sở dữ liệu thật.

6.2 Hạn chế

- Mỗi đơn hàng chỉ áp dụng được một mã giảm giá duy nhất (không hỗ trợ kết hợp nhiều mã).
- Chưa có thống kê hiệu quả sử dụng từng mã giảm giá cho quản trị viên (ví dụ tổng doanh thu đến từ các đơn có dùng mã).

6.3 Hướng phát triển

- Cho phép áp dụng đồng thời nhiều mã giảm giá không xung đột nhau.
- Gửi thông báo đẩy đến khách hàng khi có mã giảm giá mới phù hợp với lịch sử đặt hàng của họ.
- Tích hợp với chương trình tích điểm khách hàng thân thiết trong tương lai.